

DOCUMENTO DE ARQUITETURA DE SOFTWARE

Plataforma de Fluxo de Caixa Enterprise

Dimensão	Definição
Objetivo	Projetar uma solução escalável, resiliente, segura e observável para lançamentos de débito/crédito e consolidado diário.
Stack principal	C# / .NET, React + TypeScript, PostgreSQL, RabbitMQ, Redis, OpenTelemetry e Linux.
Ênfase arquitetural	Trade-offs, NFRs mensuráveis, independência de falhas, idempotência, documentação arquitetural e validação por testes.
Escopo documental	Definição da arquitetura lógica, padrões, critérios de confiabilidade, capacidade, observabilidade e evolução operacional.

Tese arquitetural

O lançamento financeiro é o caminho crítico e deve permanecer operacional mesmo quando o consolidado, o cache, a mensageria de leitura ou o assistente de voz estiverem degradados. A arquitetura foi desenhada para preservar essa independência e tornar os custos dessa decisão explícitos.

Documento técnico de arquitetura, confiabilidade, integração, segurança, capacidade e observabilidade.

1. Contexto e objetivo de negócio

O comerciante precisa registrar movimentações financeiras diárias - créditos e débitos - e consultar o saldo diário consolidado. A solução deve priorizar a continuidade do registro de lançamentos, permitir reconstrução do consolidado e apresentar comportamento previsível sob carga e falha.

Fluxos de negócio principais

- Registrar lançamento de crédito.
- Registrar lançamento de débito.
- Consultar consolidado diário.
- Visualizar saldo e composição do dia.
- Receber orientação contextual por meio de um assistente conversacional opcional, sem conceder à IA autoridade para alterar o domínio financeiro.

Princípio de arquitetura

Primeiro definimos requisitos funcionais e não funcionais; depois estabelecemos boundaries, ownership, integrações e restrições. A escolha de tecnologias é consequência desses requisitos, não o ponto de partida.

Escopo obrigatório x diferencial

Categoria	Incluído
Core obrigatório	Lançamentos, consolidado, C#, testes, documentação, resiliência, segurança e performance.
Diferencial	Frontend React, BFF, observabilidade completa, assistente 3D por voz, testes de caos controlados e dashboard de SLOs.

2. Requisitos funcionais e não funcionais

Requisitos funcionais descrevem o que o sistema faz. NFRs descrevem com quais garantias de disponibilidade, desempenho, segurança e recuperação essas funcionalidades devem operar.

Requisitos funcionais

ID	Requisito	Definição
RF-01	Cadastrar lançamento	Aceitar débito ou crédito com valor, data, descrição e identificação do comerciante.
RF-02	Consultar consolidado	Retornar saldo diário e totais de créditos/débitos.
RF-03	Idempotência	Repetir a mesma requisição com a mesma chave não pode criar novo efeito financeiro.
RF-04	Assistência	Orientar o usuário sobre navegação e uso do sistema sem executar alteração financeira autônoma.

NFRs transformados em atributos mensuráveis

Atributo	Meta definida	Racional
Disponibilidade - Lançamentos	SLO 99,95% mensal	Caminho crítico do produto.
Disponibilidade - Consolidado	SLO 99,90% mensal	Pode sofrer indisponibilidade sem interromper lançamentos.
RTO - Lançamentos	<= 5 min	Serviço crítico deve recuperar rapidamente.
RPO - Lançamentos	0 para transação confirmada	Nenhum lançamento já confirmado ao cliente pode ser perdido.
RTO - Consolidado	<= 15 min	Projeção pode ser reconstruída/reprocessada.
RPO - Consolidado	Sem perda lógica	O consolidado é uma projeção e pode ser reconstruído a partir dos lançamentos/eventos persistidos.
Capacidade - Consolidado	50 RPS obrigatório; meta interna 100 RPS	Atender a carga prevista com margem.
Latência - leitura	p95 <= 200 ms; p99 <= 400 ms	Manter experiência previsível, não apenas média baixa.
Latência - escrita	p95 <= 300 ms; p99 <= 600 ms	Confirmar persistência durável antes de devolver sucesso.
Perda de requisições	<= 5% obrigatório; objetivo <= 1%	Meta interna de confiabilidade e capacidade.
Segurança	100% HTTPS; autenticação; autorização; audit trail	Proteção transversal do dado financeiro.
Observabilidade	100% das requisições com correlation/trace id	Permitir rastreamento ponta a ponta.

Observação sobre RPO do consolidado

O consolidado não é a fonte primária da verdade. Se a base de leitura for perdida, os lançamentos e eventos persistidos permanecem autoritativos e a projeção pode ser recalculada. Isso não equivale a dizer que o banco físico nunca pode falhar; significa que não há perda lógica de negócio enquanto a fonte autoritativa estiver preservada.

3. Domínios, bounded contexts e ownership

A decomposição não foi feita por tecnologia, mas por responsabilidade e criticidade de negócio.

Bounded Context	Papel	Responsabilidade	Ownership
Lançamentos	Write Model	Créditos, débitos, idempotência e outbox.	PostgreSQL de escrita
Consolidação	Read Model	Saldo diário, totais e projeção de leitura.	PostgreSQL de leitura
Identidade e Acesso	Supporting	Autenticação, autorização, sessão e políticas.	Keycloak/claims + configuração
Assistência Conversacional	Supporting	Navegação assistida, ASR/LLM/TTS e avatar 3D.	Estado efêmero; sem ownership financeiro
Observabilidade	Platform	Logs, métricas, traces, SLOs e alertas.	Backends de telemetria

Regra de ownership

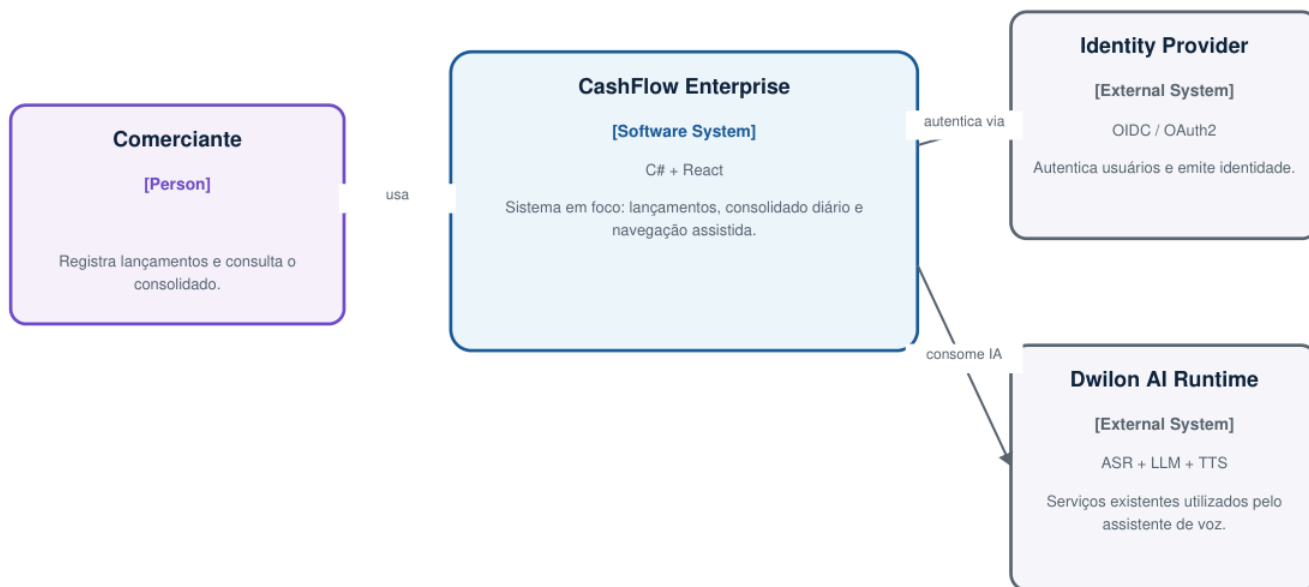
Somente o contexto de Lançamentos cria e confirma movimentações financeiras. O Consolidado possui uma projeção derivada e nunca deve se tornar a origem autoritativa. O assistente de IA não recebe acesso direto ao banco financeiro e só pode executar ferramentas explicitamente autorizadas de navegação/explicação.

Dependências permitidas

- Frontend depende do BFF/Edge, não da topologia interna dos serviços.
- Lançamentos depende apenas de seu banco transacional para confirmar a operação.
- Consolidação depende do fluxo assíncrono, mas sua queda não bloqueia escrita.
- Redis é aceleração/controle; não é mecanismo único de integridade financeira.
- Assistente conversacional é um domínio de falha separado do core financeiro.

4. Arquitetura C4 - C1: System Context

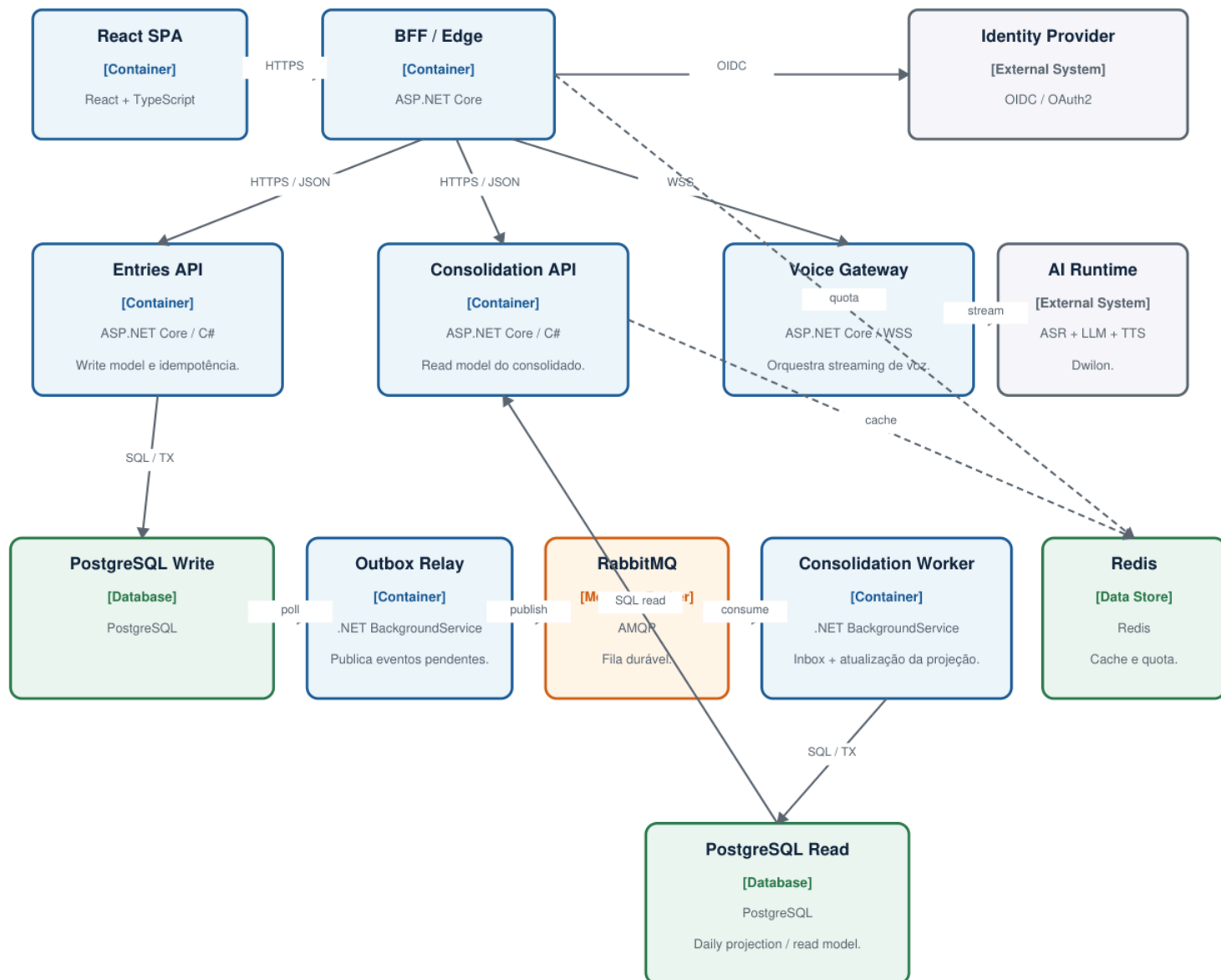
Nível 1: pessoa, sistema em foco e sistemas externos. Nenhum detalhe de container ou infraestrutura é mostrado neste nível.



Leitura do C1: o CashFlow é o sistema em foco. Identity Provider e Dwilon AI Runtime são dependências externas. A indisponibilidade da IA não deve impedir os fluxos financeiros do CashFlow.

4.1 C4 - C2: Container Diagram

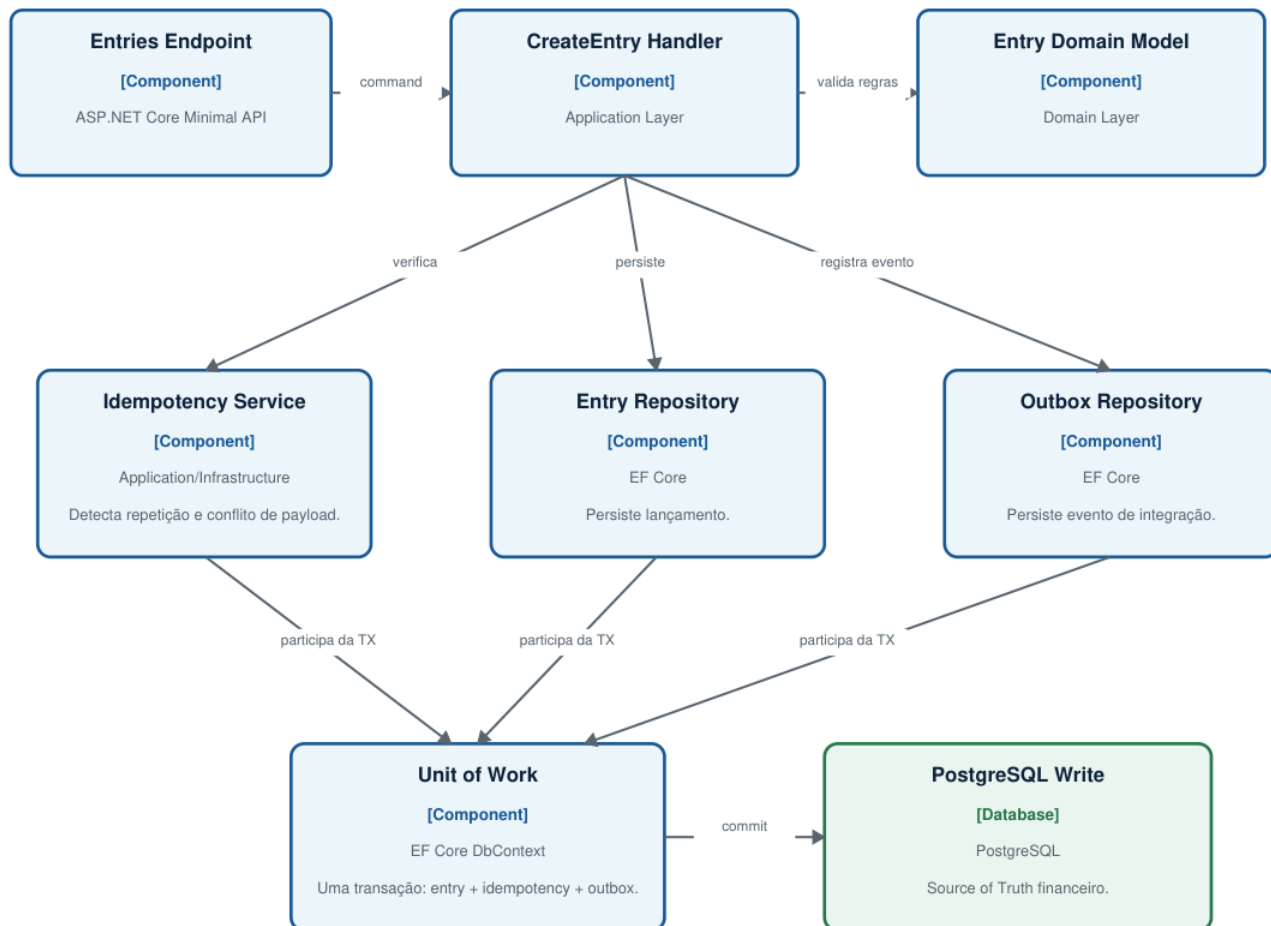
Nível 2: aplicações executáveis, data stores e message broker. Relações indicam protocolo ou propósito.



Boundary crítico: Entries API confirma a escrita sem depender de Consolidation API, RabbitMQ, Redis ou IA. RabbitMQ participa somente após a persistência durável via Outbox.

4.2 C4 - C3: Component Diagram - Entries API

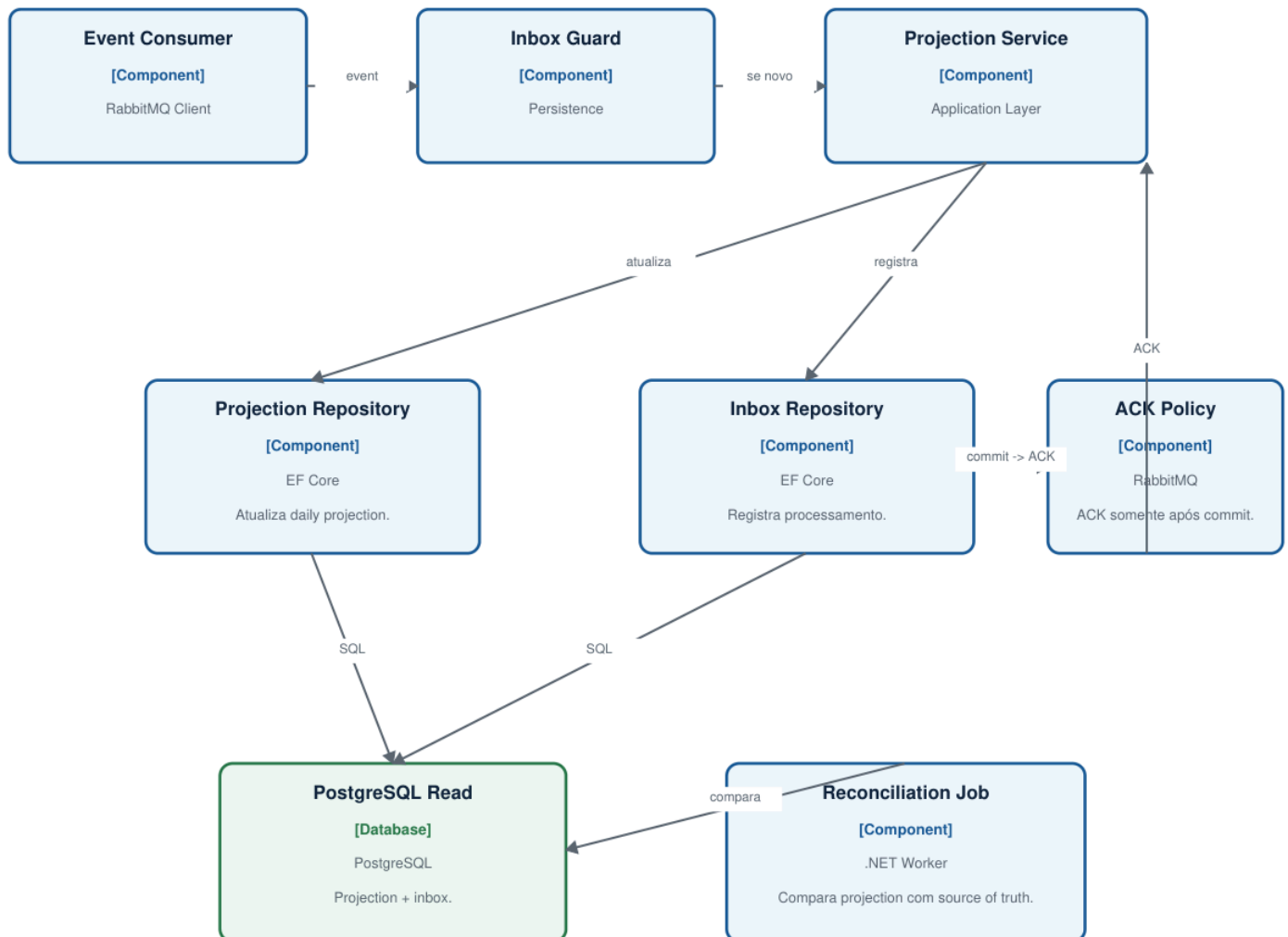
Nível 3: componentes internos do container responsável pelo caminho crítico de escrita financeira.



Regra arquitetural: HTTP 201 somente após o commit local. O broker não participa desta transação. Se RabbitMQ estiver indisponível, o evento permanece na Outbox e será publicado posteriormente.

4.3 C4 - C3: Component Diagram - Consolidation Worker

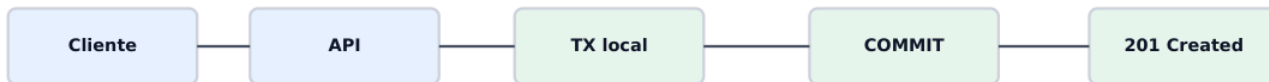
Nível 3: componentes que garantem consumo idempotente, atualização da projeção e capacidade de reconciliação.



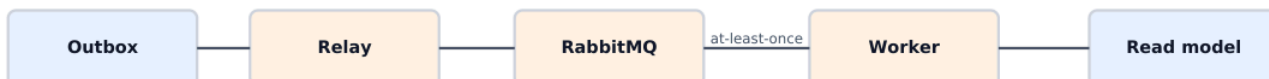
A solução assume reentrega. O efeito financeiro permanece idempotente porque Inbox e projeção são persistidos de forma transacional antes do ACK. Reconciliação adiciona uma camada de controle de integridade.

5. Fluxos síncronos e assíncronos

Caminho síncrono (confirmação do lançamento)



Caminho assíncrono (consolidação)



Fluxo síncrono de lançamento

- Cliente envia POST com Idempotency-Key.
- API valida autorização e contrato.
- Uma transação local grava lançamento, proteção de idempotência e registro de outbox.
- Somente após COMMIT durável a API retorna 201 Created.
- Falha antes do commit resulta em erro explícito; não existe sucesso fictício.

Fluxo assíncrono de consolidação

- Outbox Relay lê eventos pendentes e publica no RabbitMQ com confirmação.
- RabbitMQ entrega pelo menos uma vez ao Consolidation Worker.
- Worker grava inbox/deduplicação e atualiza a projeção diária na mesma transação.
- ACK ocorre após commit. Reentrega é tolerada por idempotência.
- Mensagens irrecuperáveis seguem para DLQ com observabilidade e política de reprocessamento.

Garantia que NÃO é prometida

A arquitetura não promete exactly-once ponta a ponta. A garantia prática é at-least-once delivery com efeito idempotente no consumidor. Essa formulação é tecnicamente mais defensável em sistemas distribuídos.

6. Trade-offs arquiteturais

Cada decisão foi registrada com benefício, custo assumido, mitigação e evidência de validação.

Decisão	Escolha	Ganho	Custo	Mitigação
Síncrono x assíncrono	Escrita durável síncrona; efeitos derivados assíncronos	Independência e absorção de picos	Consistência eventual e maior complexidade	Idempotência, outbox/inbox, tracing, DLQ
Bancos separados	Write e read model independentes	Isolamento de falhas e escala de leitura	Sem transação global	Reconciliação e replay
Redis como cache	Cache apenas no read path	Baixa latência e menos carga	Stale data / invalidation	TTL curto e fallback para banco
RabbitMQ	Fila durável para consolidação	Buffer, retry e desacoplamento temporal	Operação de broker e reentrega	Publisher confirms, ACK, DLQ, métricas
React SPA	SPA autenticada	Simplicidade e boa experiência interativa	Sem benefício de SSR/SEO	SEO não é requisito do produto
Assistente 3D	Fora do critical path	Diferencial de experiência	Mais dependências e latência	Circuit breaker/timeout; degradação sem impacto financeiro
Compose no teste	Deploy simples em Linux	Reprodutibilidade	Host continua SPOF	Documentar evolução multi-host

Frase de defesa

Uma boa decisão arquitetural não é aquela sem desvantagens; é aquela em que sabemos quais desvantagens estamos aceitando, por que são aceitáveis e como vamos medir se a escolha continua válida.

7. ADRs - Architecture Decision Records

As decisões relevantes ficam versionadas no repositório para preservar contexto, alternativas e consequências. Abaixo está o resumo dos ADRs centrais.

ADR	Decisão	Razão principal
ADR-001	Separar write model e read model	Preservar lançamentos quando consolidado falhar.
ADR-002	Outbox transacional	Evitar dual-write entre banco e broker.
ADR-003	Inbox/idempotência no consumidor	Tolerar reentrega sem duplicar saldo.
ADR-004	RabbitMQ para consolidação	Fila durável e padrão de trabalho assíncrono do case.
ADR-005	Redis não autoritativo	Cache e quota não podem ser a única garantia financeira.
ADR-006	BFF em ASP.NET Core	Isolar frontend da topologia e centralizar edge concerns.
ADR-007	Assistente sem escrita financeira	Conter risco de um componente não determinístico.
ADR-008	OpenTelemetry como padrão	Uniformizar logs, métricas e traces.
ADR-009	Docker Compose no laboratório	Reprodutibilidade sem alegar HA física.
ADR-010	PostgreSQL como fonte de verdade	Transação, integridade e simplicidade operacional.

Estrutura esperada de cada ADR

Contexto -> decisão -> alternativas consideradas -> consequências positivas -> consequências negativas -> sinais para reavaliar a decisão.

8. Consistência, idempotência e recuperação

A estratégia parte do princípio de que falhas e reentregas são normais em sistemas distribuídos.

Idempotência de comando

O cliente envia uma chave idempotente. A primeira requisição cria o lançamento e associa a chave ao fingerprint do payload. Repetições com a mesma chave e mesmo payload retornam o resultado original; a mesma chave com payload diferente retorna conflito.

Idempotência de evento

Cada evento possui event_id único. O consumidor persiste a entrada na inbox dentro da mesma transação que atualiza o read model. Uma reentrega do mesmo event_id é reconhecida e não altera novamente o saldo.

Rebuild do consolidado

O consolidado é uma projeção. Em caso de perda da base de leitura, a aplicação consegue reconstruir o estado a partir dos lançamentos/eventos persistidos. Por isso o RPO é descrito como sem perda lógica, e não como impossibilidade de perda física do banco de leitura.

Exemplo

Crédito +1000 Débito -200 Crédito +500 Read model perdido -> replay/reconciliação -> saldo reconstruído = 1300

Reconciliação

Além do replay, um job de reconciliação compara quantidade e soma dos lançamentos autoritativos com a projeção diária. Divergências geram alerta e podem iniciar reparo controlado.

9. Segurança e proteção do edge

Segurança é tratada como requisito arquitetural, não como etapa posterior.

Controle	Aplicação
Autenticação	OIDC/OAuth2 por provedor de identidade; tokens de curta duração.
Autorização	RBAC/claims por operação e tenant.
Transporte	HTTPS/WSS; TLS obrigatório no edge.
Segredos	Variáveis/secret store; nunca versionados no Git.
Validação	DTOs estritos, limites de tamanho, normalização e validação de entrada.
Rate limit	Política por identidade/rota; não usada para mascarar incapacidade do benchmark.
Auditoria	Registro de criação de lançamento, identidade e correlation id.
IA	Allowlist de ferramentas; sem SQL, shell, escrita financeira ou execução arbitrária.

Rate limit x capacidade

O cenário obrigatório de 50 RPS deve ser aceito pelo sistema. Respostas 429 durante esse cenário contam como falha de capacidade. O teste de rate limit é separado e excede deliberadamente a cota para comprovar proteção contra abuso sem degradar outros clientes.

10. Observabilidade, SLI/SLO e operação

Todos os serviços instrumentam logs estruturados, métricas e traces com OpenTelemetry.

Área	SLIs / métricas
HTTP	request rate, error rate, p50/p95/p99, 4xx/5xx
Lançamentos	commit latency, conflitos de idempotência, taxa de sucesso
Outbox	eventos pendentes, idade do evento mais antigo, falhas de publicação
RabbitMQ	queue depth, publish/ack rate, redeliveries, DLQ
Consolidado	projection lag, throughput, divergências de reconciliação
Redis	hit ratio, latência, erros e fallback
Voice	ASR latency, LLM TTFT, TTS first-audio, sessão e cancelamentos
Infra	CPU, memória, disco, conexões, restart count

Golden signals aplicados

- Latency: p50, p95 e p99; não apenas média.
- Traffic: RPS/TPS e throughput por componente.
- Errors: erro técnico e resposta inválida de negócio.
- Saturation: CPU, memória, pool de conexões, fila e recursos de inferência.

Alertas principais

- SLO burn rate acima do orçamento de erro.
- Projection lag acima de 5 s por janela sustentada.
- Outbox mais antiga acima de 30 s.
- DLQ > 0.
- Reconciliação divergente.
- p99 excedendo meta por período sustentado.

11. Critérios de Performance e Capacidade

Os valores abaixo definem metas técnicas e critérios de validação para capacidade, latência e confiabilidade.

Cenário A - Consolidado, requisito obrigatório

Métrica	Critério de referência	Meta	Avaliação
Carga	50 RPS por 10 min (30.000 requests)	50 RPS	Obrigatório
Sucesso	99,99%	? 95% obrigatório / ? 99% objetivo	Atende meta
p50	18 ms	Informativo	Referência
p95	47 ms	? 200 ms	Atende meta
p99	82 ms	? 400 ms	Atende meta
5xx	0,01%	? 1% objetivo	Atende meta

Cenário B - Margem de capacidade

Métrica	Critério de referência	Avaliação
100 RPS sustentados / 10 min	99,95% sucesso; p95 91 ms; p99 164 ms	Atende meta
Burst 200 RPS / 60 s	99,72% sucesso; p95 176 ms; p99 338 ms	Atende meta
CPU app média / pico	38% / 67%	Dentro do envelope
DB connections pico	42% do pool	Dentro do envelope

Cenário C - Escrita e idempotência

Teste	Critério de referência
15.000 lançamentos	15.000 confirmados; 0 perdidos
p50 / p95 / p99	31 ms / 86 ms / 143 ms
100 requisições concorrentes, mesma Idempotency-Key	1 lançamento criado; respostas consistentes com o resultado original
Mesma chave com payload diferente	Conflito explícito; nenhum novo efeito financeiro

12. Plano de Resiliência e Recuperação

A arquitetura é validada nas janelas de falha que justificam seus patterns e mecanismos de recuperação.

Falha injetada	Resultado esperado	Evidência esperada
Consolidado indisponível por 5 min	Lançamentos permanecem disponíveis; nenhum dado confirmado é perdido.	Independência do write path.
Backlog após retorno	Eventos pendentes são drenados; projeção converge sem duplicidade.	Recuperação do read model.
Publisher cai após publish antes de marcar Outbox	Evento pode ser reenviado; consumidor produz um único efeito.	Tolerância à ambiguidade de publicação.
Worker cai após COMMIT antes de ACK	RabbitMQ reentrega; saldo permanece correto.	Inbox + idempotência.
Redis indisponível	Leituras usam fallback definido; core financeiro permanece disponível.	Cache não é dependência crítica.
Assistente de voz indisponível	Core financeiro permanece funcional.	IA fora do critical path.
Rate limit abusivo	Ator acima da quota recebe 429; tráfego legítimo preserva SLO.	Isolamento no edge.

Limites da topologia

Executar todos os containers em um único host Linux oferece isolamento de processos e recuperação de componentes, mas não entrega alta disponibilidade contra falha física do host. A evolução operacional exige múltiplos nós ou zonas, réplica de dados, balanceamento e testes periódicos de failover.

13. Frontend e assistente conversacional 3D

O frontend React + TypeScript é uma SPA autenticada orientada a tarefas: dashboard, lançamentos, consolidado, arquitetura/observabilidade da demo e assistência contextual.

Componentes principais

- React Router para navegação.
- TanStack Query para server state e cache de chamadas HTTP.
- Estado local/global mínimo para experiência do usuário.
- Three.js / React Three Fiber para avatar feminino 3D.
- Web Audio API para captura de microfone mediante consentimento.
- WebSocket seguro para sessão de voz e streaming de áudio.

Pipeline da assistência

Microfone -> Voice Gateway C# -> ASR existente -> LLM existente -> política/tool validation -> TTS existente -> áudio no browser -> animação do avatar.

Boundary de segurança

A IA pode explicar e navegar. Ela não pode criar, alterar ou excluir lançamentos financeiros. A execução de uma ação depende de tools pré-definidas e validadas pelo backend. Isso mantém um componente probabilístico fora da autoridade transacional.

Trade-off da voz

No pipeline conversacional, latência pesa mais do que durabilidade de fila. Por isso a interação de voz é stream-oriented e cancelável. Já a consolidação financeira prioriza recuperação, reentrega e durabilidade. A mesma arquitetura não é aplicada cegamente a domínios com NFRs diferentes.

14. Estratégia de testes

A pirâmide de testes protege regra, integração, contrato, experiência e atributos de qualidade.

Tipo	Objetivo	Ferramenta
Unitário	Regras de domínio, cálculo, validação e idempotência pura.	xUnit / FluentAssertions
Integração	PostgreSQL, RabbitMQ, Redis, transações e migrations.	Testcontainers
Contrato	OpenAPI/event schema e compatibilidade entre producer/consumer.	Schema tests
E2E	Fluxos essenciais pelo browser.	Playwright
Carga	50/100/200 RPS e percentis.	k6
Mutação	Efetividade da suíte de unidade.	Stryker.NET
Resiliência	Kill/restart de serviços e broker/clientes.	Docker + scripts controlados
Reconciliação	Comparar source of truth e read model.	Testes automatizados / job

Critérios de aceitação técnica

- ? Build reproduzível.
- ? Testes automatizados verdes.
- ? 50 RPS por 10 min dentro do limite de erro definido.
- ? Idempotência validada sob concorrência.
- ? Consolidado pode falhar sem interromper lançamentos.
- ? Read model converge após recuperação.
- ? README permite execução local/Linux sem conhecimento tácito.

15. Implantação Linux e Topologia Operacional

Os serviços são empacotados em containers e executados em Linux por Docker Compose, com redes, volumes e responsabilidades segregadas.

frontend, bff, launch-api, consolidated-api, outbox-relay, consolidation-worker, postgres-write, postgres-read, rabbitmq, redis-cache, redis-control, voice-gateway, otel-collector, prometheus, grafana, loki, tempo, keycloak.

Princípios operacionais

- Health/readiness separados de liveness.
- ? Limites de CPU/memória por container.
- Volumes persistentes para bancos/broker quando aplicável.
- Somente edge exposto externamente; bancos não são publicados.
- Configuração por ambiente; secrets fora do repositório.
- Migrations controladas e executadas como etapa explícita de deploy.
- Shutdown gracioso para workers e consumidores.

Evolução para produção enterprise

A arquitetura lógica permite migrar de Compose para um scheduler/orquestrador apenas quando os requisitos de disponibilidade e operação justificarem. A produção real poderia adicionar múltiplos nós, balanceadores, banco gerenciado/replicado, broker em cluster, object storage para backups e segregação de workloads de IA.

16. SLI, SLO e Error Budget

Os objetivos de confiabilidade só são úteis quando existe uma forma objetiva de medi-los. Por isso cada SLO relevante possui um SLI associado.

SLIs principais

SLI	Medição	SLO / expectativa
Disponibilidade de lançamentos	successful_write_requests / valid_write_requests	SLO >= 99,95%
Disponibilidade do consolidado	successful_read_requests / valid_read_requests	SLO >= 99,90%
Latência do consolidado	http.server.request.duration	p95 <= 200 ms; p99 <= 400 ms
Lag de projeção	now - event_applied_at	p95 <= 2 s; p99 <= 5 s
Saúde da Outbox	idade do evento pendente mais antigo	normalmente próxima de zero; alerta por janela
Integridade	diferença entre source of truth e read model	0 após reconciliação

Error budget

Para SLO mensal de 99,95%, o orçamento teórico de indisponibilidade é 0,05% do período, aproximadamente 21 min 36 s em um mês de 30 dias. Se o consumo do budget acelerar, a prioridade operacional migra de novas mudanças para confiabilidade.

Princípio

SLI é o que medimos; SLO é a meta; SLA é um compromisso formal externo. Neste documento são definidos SLIs e SLOs técnicos, não um SLA comercial.

17. Architecture Fitness Functions

Algumas regras arquiteturais são transformadas em testes automáticos. O objetivo é evitar que a arquitetura exista apenas no diagrama e seja gradualmente violada pelo código.

Regras executáveis propostas

Regra	Validação
Domain não depende de Infrastructure	Teste de referência entre assemblies/namespaces
Lançamentos não depende de Consolidação	Boundary test no build
Voice Assistant não referencia Write Application	Regra de dependência proibida
Nenhum secret no Git	Secret scanning no CI
Contratos OpenAPI/eventos são válidos	Lint + contract tests
Imagem sem vulnerabilidade crítica aceita	Container/dependency scanning
Migrations não são disparadas implicitamente por qualquer replica	Etapa controlada de deploy

Valor arquitetural

A fitness function converte uma decisão de arquitetura em uma propriedade continuamente verificada. Se um commit violar um boundary crítico, o pipeline falha antes do deploy.

18. Alternativas Avaliadas e Deliberadamente Rejeitadas

Arquitetura enterprise também envolve saber quando não introduzir uma tecnologia. As rejeições abaixo reduzem complexidade sem sacrificar os requisitos da solução.

Alternativa	Motivo da rejeição
Kafka	Não há requisito de replay histórico massivo ou throughput que justifique seu custo operacional. RabbitMQ atende fila, ack, retry e DLQ do fluxo de consolidação.
Kubernetes neste estágio	Um cluster sobre um único host não remove o SPOF físico. Compose demonstra o desenho com menor custo operacional; HA real fica no roadmap multi-host.
Event Sourcing completo	Reconstruir o read model não exige transformar todo o estado de negócio em event stream.
MongoDB como base principal	O domínio favorece integridade transacional, constraints e consistência relacional.
Redis como source of truth	Cache e controle efêmero não devem ser a única proteção de integridade financeira.
Dezenas de microsserviços	O domínio é pequeno; boundaries são criados apenas quando existe diferença real de ownership, escalabilidade ou falha.
IA com acesso financeiro	Um componente probabilístico não recebe autoridade para criar, excluir ou estornar lançamentos.

Critério

Tecnologia não justifica tecnologia. Toda complexidade adicional precisa responder a um requisito ou risco concreto.

19. CI/CD, qualidade e supply chain

O pipeline aplica o princípio build once, promote many: o mesmo artefato aprovado é promovido entre ambientes, reduzindo divergência e aumentando rastreabilidade.

Etapa	Gate
1. Restore / Build	Build reproduzível e warnings críticos tratados.
2. Unit / Architecture Tests	Regra de negócio e boundaries.
3. Integration / Contract	PostgreSQL, RabbitMQ, Redis e contratos.
4. Mutation	Qualidade dos testes em módulos críticos.
5. SAST / Secrets / Dependencies	Gate de segurança de código e dependências.
6. Container Build / Scan	Imagem versionada e sem vulnerabilidade crítica aceita.
7. Deploy Test	Migrations controladas + health/readiness.
8. Smoke / E2E	Fluxos principais do produto.
9. Performance	Carga e regressão de p95/p99 quando aplicável.

Rastreabilidade

Cada imagem/artefato é associado ao commit, versão do schema e resultado de testes. Rollback e roll-forward são decisões explícitas; migrations destrutivas exigem compatibilidade temporal e plano de reversão.

20. Matriz de Riscos Arquiteturais

Os principais riscos são conhecidos antes da implementação e possuem mitigação e sinal operacional associado.

Risco	Impacto	Mitigação	Sinal
Entries DB indisponível	Crítico	HA/backup/failover; health checks; recovery test	write availability
Broker indisponível	Médio imediato	Outbox preserva intenção de publicação	outbox_oldest_age
Consumer indisponível	Baixo imediato	Fila durável + recovery	queue depth / lag
Evento duplicado	Alto	Inbox + unique constraints + idempotência	redelivery / duplicate count
Poison message	Médio	Retry finito + DLQ	dlq_total
Projection divergente	Alto	Reconciliação + replay	reconciliation_difference
Redis indisponível	Médio	Fallback e degradação controlada	cache errors / fallback
IA indisponível	Baixo	Isolamento do core financeiro	voice availability
Perda do host	Alto	Não resolvido nesta topologia; evolução multi-host.	infra availability

Princípio

Um risco não desaparece porque existe um pattern no diagrama. Ele precisa de mecanismo de mitigação, telemetria e teste de recuperação.

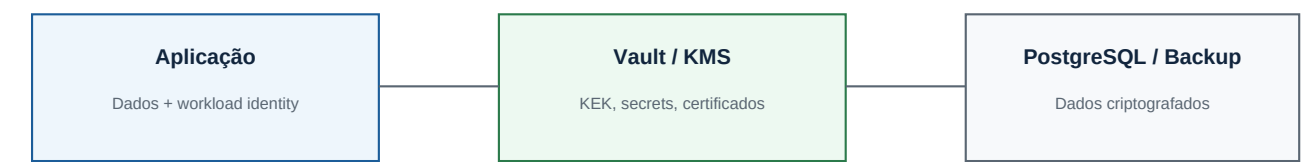
21. Security, Privacy & Compliance Architecture

Segurança por design, proteção de dados, identidade, criptografia e controles de conformidade.

Princípio: nenhum dado, segredo, workload ou segmento de rede é considerado confiável apenas por estar dentro do ambiente. Identidade, autorização, criptografia, rastreabilidade e minimização são aplicadas em camadas.

Domínio	Controle enterprise	Aplicação na solução
LGPD / Privacy by Design	Finalidade, minimização, retenção, descarte e direitos do titular	PII coletada somente quando necessária; retenção e exclusão controladas.
Data Classification	Público / interno / confidencial / restrito	Dados financeiros e identificadores pessoais classificados como restritos.
IAM	OIDC/OAuth2 + MFA + RBAC	Usuários humanos e workloads possuem identidades distintas e menor privilégio.
Secrets	HashiCorp Vault	Credenciais, tokens, certificados e chaves fora do código e das imagens.
Transit Encryption	TLS 1.3 / mTLS onde aplicável	Browser->Edge via TLS; serviços críticos podem usar mTLS.
At-rest Encryption	Volumes/backups criptografados + criptografia de campo quando necessária	PII crítica protegida também na camada de aplicação.
Audit	Trilha imutável de eventos administrativos/financeiros	Quem, quando, qual ação, resultado e correlation_id.

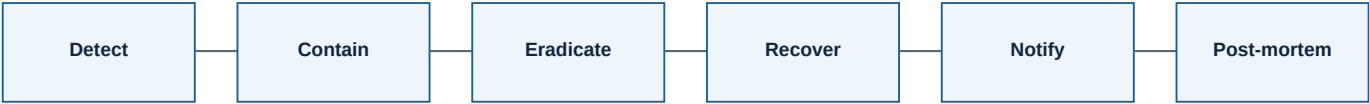
Modelo de proteção de dados e chaves



Controles de execução, rede, pipeline, backups, auditoria e tratamento de incidentes.

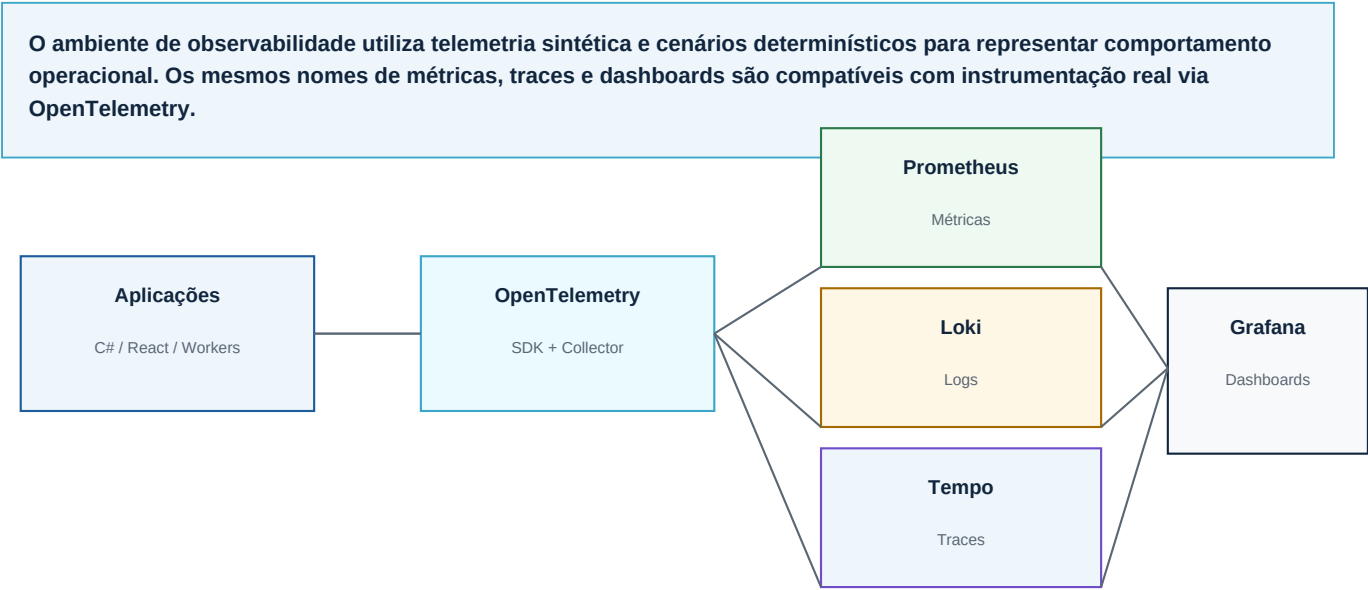
Área	Definição
Workload Identity	entries-api, consolidation-worker, outbox-relay e voice-gateway usam identidades próprias; sem credencial administrativa compartilhada.
Segregação de rede	Edge, Application, Messaging, Data, AI e Management em zonas lógicas distintas; bancos e painéis administrativos não são públicos.
PostgreSQL	TLS, usuário por workload, sem superuser, grants mínimos, backup criptografado e restore periódico.
RabbitMQ	TLS, vhost dedicado, usuário por aplicação, permissões por exchange/queue e management restrito.
Redis	ACL/TLS, rede privada, sem uso como source of truth financeiro.
DevSecOps	SAST, secret scan, dependency/IaC/container scan, SBOM e policy gates no CI/CD.
Supply Chain	Artefatos versionados, imagens assinadas e verificação de proveniência antes do deployment.
Logs e LGPD	PII redigida/mask; tokens e Authorization nunca persistidos em logs; acesso e retenção restritos.
Backups	Criptografados, off-host, retenção definida, integridade verificada e restore efetivamente testado.

Fluxo de resposta a incidente



22. Observability & Reliability Control Center

Plano de monitoramento integral do sistema com telemetria sintética coerente e contratos compatíveis com produção.

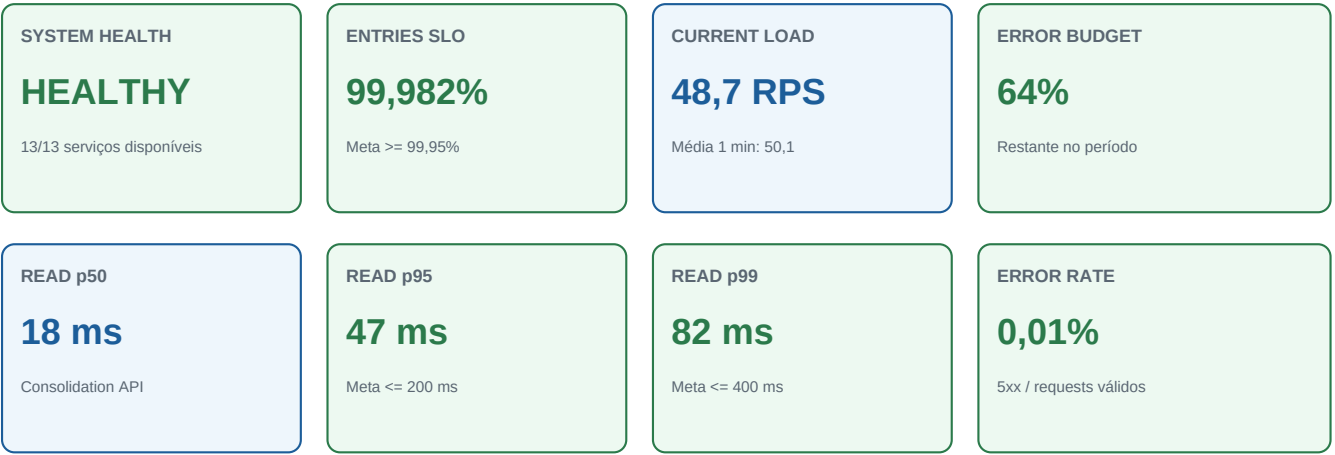


Telemetria padronizada

Classe	Métricas / atributos
HTTP	http_server_request_duration_seconds, requests_total, status_code, route
Financeiro	cashflow_entries_created_total, entries_failed_total, daily_balance
Outbox	cashflow_outbox_pending, cashflow_outbox_oldest_age_seconds
Mensageria	rabbitmq_queue_messages_ready, publish_rate, consume_rate, ack_rate
Consolidação	cashflow_projection_lag_seconds, reconciliation_difference
Cache	redis_cache_hit_ratio, redis_latency, evictions
Infra	process_cpu_utilization, process_memory_usage_bytes, connections, disk I/O
Tracing	trace_id, span_id, correlation_id, event_id, idempotency_key

22.1 Dashboard Operacional - Estado Sintético de Referência

Visão única para saúde dos serviços, SLO, latência, filas, dados, infraestrutura, segurança e IA.



Saúde por serviço

React SPAUP	BFF / EdgeUP	Entries APIUP
PostgreSQL WriteUP	Outbox RelayUP	RabbitMQUP
Consolidation WorkerUP	PostgreSQL ReadUP	Consolidation APIUP
RedisUP	Voice GatewayUP	ASR / LLM / TTSUP

Filas, banco, cache e IA

Indicador	Valor sintético	Estado
Outbox pending	0	Healthy
RabbitMQ ready / unacked	0 / 11	Healthy
Projection lag p95 / p99	1,2 s / 2,8 s	Healthy
PostgreSQL Write query p95	11 ms	Healthy
Redis hit ratio	94,7%	Healthy
Voice ASR / LLM TTFT / TTS TTFA p95	181 / 328 / 218 ms	Healthy
Voice E2E p95	1,08 s	Healthy

22.2 Synthetic Scenario Engine e Failure Simulation

Mecanismo determinístico para simular carga, degradação, falhas e recuperação mantendo causalidade entre as métricas.

Cenário	Mudanças causais esperadas
NORMAL	50 RPS aproximados; p95 baixo; fila próxima de zero; serviços UP.
LOAD_100	RPS e CPU aumentam; p95/p99 sobem de forma moderada; error rate permanece dentro do SLO.
SPIKE_200	Pico temporário de CPU, conexões e latência; recuperação após o burst.
WORKER_DOWN	Worker DOWN; queue depth e projection lag crescem; Entries API permanece HEALTHY.
BROKER_DOWN	RabbitMQ DOWN; Outbox pending/oldest age crescem; escrita financeira continua localmente.
REDIS_DOWN	Cache DOWN; Consolidation API DEGRADED; fallback ao PostgreSQL Read aumenta p95.
DUPLICATE_EVENT	Reentrega ocorre; Inbox bloqueia novo efeito; saldo permanece estável.
RECOVERY	Consumer throughput supera publish rate; backlog e lag convergem até o estado saudável.

Exemplo de comportamento correlacionado - WORKER_DOWN

Entries API

UP

estado sintético

Queue

0 -> 1.240

estado sintético

Lag p95

1,2s -> 32s

estado sintético

Worker

DOWN

estado sintético

A simulação não usa números independentes por card. Cada cenário altera um estado global de carga e falha; RPS, latência, CPU, filas, lag e disponibilidade são derivados desse estado para preservar coerência operacional.

23. Conclusão Executiva

A solução foi desenhada a partir das garantias exigidas pelo negócio, e não a partir de uma lista de tecnologias.

Princípios consolidados

- ? NFRs convertidos em SLOs, RTO/RPO, percentis e metas de capacidade.
- ? Bounded contexts e ownership explícitos.
- ? Trade-off consciente entre consistência imediata e independência do consolidado.
- ? Idempotência e Outbox/Inbox para lidar com falhas realistas.
- ? Cache e IA fora da fonte de verdade financeira.
- ? Observabilidade projetada como parte da arquitetura.
- ? Testes de carga, falha e recuperação vinculados às decisões arquiteturais.
- ? Limites da topologia single-host explicitados sem tratá-la como alta disponibilidade.

Princípio arquitetural

A arquitetura não busca maximizar simultaneamente simplicidade, disponibilidade, consistência, performance e custo. Ela prioriza os atributos mais relevantes para cada fluxo, explicita o custo de cada decisão e define como verificar se o trade-off permanece adequado ao longo da evolução do sistema.

Os indicadores de capacidade, confiabilidade e recuperação devem ser atualizados automaticamente a partir de execuções controladas, telemetria operacional e evidências versionadas no pipeline.